
 Tuesday, January 11, 2022 12:52 PM

Verilog Syntax

```
wire p0;
```

```
Wire [7:0] data1, data2; data bus of width 8 bit
```

```
Reg [3:0] mem [31:0] // 32 locations memory with 4-bit words
```

```
// 1-bit comparator
```

```
i0, i1 ==> eq
```

```
Module onebitcompare (i0, i1, eq);
```

```
    Input i0, i1;
```

```
    Output eq;
```

```
    Wire p0, p1;
```

```
    Assign p1 = i0 & i1 ; // logical AND
```

```
    Assign p0 = ~i0 & ~i1; // logical AND
```

```
    Assign eq = p0 | p1 ; // logical OR
```

```
Endmodule
```

Instantiate modules

```
a1a0, b1b0; =?
```

```
onebitcompare comp_lsb(a0, b0, e0);
```

```
onebitcompare comp_msb(a1, b1, e1);
```

```
Assign out = e0 & e1; // AND
```

```
// mapping between parameters
```

```
onebitcompare comp_lsb( .i0(a0), .eq(e0), .i1(b0));
```

Verilog : structural + behavioural

Behavioural

sum = a+b, a-b, high-level

Comparison >, <, =

Structural = gate level

```
Module struc_compare(input wire i0, i1, output wire eq);
```

```
wire p0, p1, i0bar, i1bar;
```

```
Not gate1(i0bar, i0);
```

```
Not gate2(i1bar, i1);
```

```
and gate3(p0, i0, i1);
```

```
and gate4(p1, i0bar, i1bar);
```

```
Or gate5(eq, p0, p1);
```

```
Endmodule
```

Always // used to trigger a set of stmts

always @(x) // when x changes do something

```
always @(posedge clk) // clk variable
begin
End
```

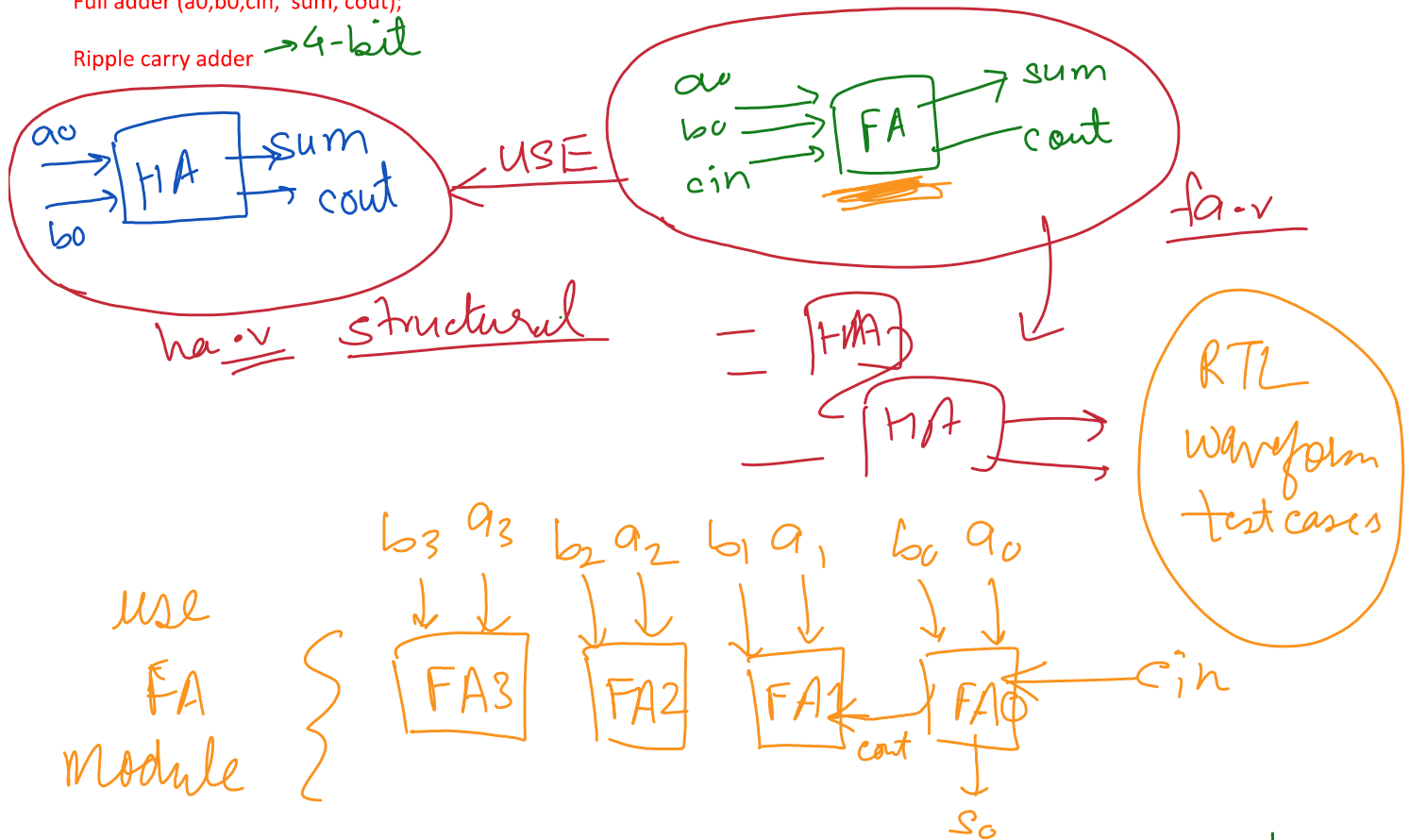
```
always @*
begin
  If( r[4] == 1'b1)
    Y = 3'b100;
  Else if(r[3] == 1'b1)
    Y = 3'b011;
  ...
End
```

ADDERS

Half adder (a0,b0,sum, cout);

Full adder (a0,b0,cin, sum, cout);

Ripple carry adder → 4-bit



H/A, FA → 3:15 → 4:00 pm } submit TA

RCA → 4:00 - 5:00 pm }

